

Sistema de Base de Datos para Restaurante

Documentación Técnica Completa

Proyecto Final — Bases de Datos

Grupo 01

Motor: PostgreSQL 18

Fecha: Mayo 2026

Scripts: 6 archivos SQL

Índice

1. Descripción General del Proyecto	2
1.1. Orden de ejecución en PostgreSQL	2
2. Diseño del Esquema — Estrategia Tabla por Subclase	2
2.1. Tablas del sistema	3
3. 02_secuencias.sql — Secuencias	3
3.1. Descripción	3
3.2. Explicación línea por línea	3
3.3. Resumen de secuencias	4
4. 01_ddl_creacion_tablas.sql — Tablas	4
4.1. Descripción	4
4.2. Limpieza previa	4
4.3. Tablas de empleados	5
4.4. Tablas de productos	7
4.5. Tablas de órdenes	8
5. 04_funcion_folio.sql — Función generar_folio	9
5.1. Descripción	9
5.2. Explicación línea por línea	9
5.3. Ejemplos de salida	10
6. 03_triggers.sql — Triggers	10
6.1. Descripción general	10
6.2. Trigger 1 — Folio automático	10
6.3. Trigger 2 — Insertar en DETALLE_ORDEN	11
6.4. Trigger 3 — Actualizar en DETALLE_ORDEN	12
6.5. Trigger 4 — Eliminar de DETALLE_ORDEN	13
7. test_triggers.sql — Casos de Prueba	13
7.1. Descripción	13
7.2. Datos base	13
7.3. Pruebas	14
7.4. Resumen de pruebas	16
8. Diferencias entre la versión Oracle y PostgreSQL	16

Descripción General del Proyecto

Este proyecto implementa el esquema de base de datos para el sistema de gestión de un restaurante. Cubre la administración de empleados (cocineros, meseros y administrativos), catálogo de productos (platillos y bebidas), clientes con facturación, órdenes y sus detalles.

El proyecto está dividido en **seis archivos SQL**, cinco orientados a **PostgreSQL** y uno a **Oracle 23ai Free**:

Archivo	Motor	Propósito
00_ejecutar_todo.sql	Oracle 23ai	Script maestro completo
01_ddl_creacion_tablas.sql	PostgreSQL	Creación de tablas
02_secuencias.sql	PostgreSQL	Definición de secuencias
03_triggers.sql	PostgreSQL	Lógica de negocio con triggers
04_funcion_folio.sql	PostgreSQL	Función generadora de folios
test_triggers.sql	PostgreSQL	Casos de prueba

Orden de ejecución en PostgreSQL

1. 02_secuencias.sql
2. 01_ddl_creacion_tablas.sql
3. 04_funcion_folio.sql
4. 03_triggers.sql
5. test_triggers.sql

Las secuencias se ejecutan primero porque las tablas las referencian mediante `DEFAULT nextval(...)`.

Diseño del Esquema — Estrategia Tabla por Subclase

Se utiliza la estrategia *tabla por subclase* (*table per subclass*) para modelar la especialización de EMPLEADO y PRODUCTO.

- Cada subtipo tiene su propia tabla con una FK que apunta a la tabla padre.
- Permite que un empleado tenga múltiples roles simultáneamente (**especialización solapada**).
- Evita redundancia de datos y facilita la integridad referencial.

Tablas del sistema

Grupo	Tabla	Descripción
Empleados	EMPLEADO	Entidad padre con datos generales
	COCINERO	Subtipo: agrega especialidad
	MESERO	Subtipo: agrega horario
	ADMINISTRATIVO	Subtipo: agrega rol
Extras	TELEFONO_EMPLEADO	Atributo multivaluado de empleado
	DEPENDIENTE	Familiares del empleado
Productos	PRODUCTO	Entidad padre con datos generales
	PLATILLO	Subtipo de producto
	BEBIDA	Subtipo de producto
Catálogo	CATEGORIA	Clasificación de productos
Órdenes	ORDEN	Orden atendida por un mesero
	DETALLE_ORDEN	Renglones de la orden
Facturación	CLIENTE_FACT	Clientes que solicitan factura

02__secuencias.sql — Secuencias

Descripción

Define cuatro secuencias autoincrementales sin ciclo. Cada una genera llaves artificiales para una tabla específica. Se ejecuta **antes** que el DDL de tablas porque estas las referencian.

Explicación línea por línea

```

1 DROP SEQUENCE IF EXISTS folio_seq;
2
3 CREATE SEQUENCE folio_seq
4     START WITH 1          -- inicia en 1
5     INCREMENT BY 1        -- sube de uno en uno
6     MINVALUE 1           -- no puede bajar de 1
7     NO MAXVALUE           -- sin limite superior
8     NO CYCLE;             -- no reinicia al llegar al maximo

```

Listing 1: Secuencia para folios de orden

Genera los números usados para construir folios tipo ORD-001. El `NO CYCLE` es importante: garantiza que nunca se repita un folio aunque la secuencia llegue a valores muy altos.

```

1 DROP SEQUENCE IF EXISTS dependiente_id_seq;
2 CREATE SEQUENCE dependiente_id_seq
3     START WITH 1 INCREMENT BY 1 MINVALUE 1 NO MAXVALUE NO CYCLE;
4

```

```

5 DROP SEQUENCE IF EXISTS categoria_id_seq;
6 CREATE SEQUENCE categoria_id_seq
7     START WITH 1 INCREMENT BY 1 MINVALUE 1 NO MAXVALUE NO CYCLE;
8
9 DROP SEQUENCE IF EXISTS producto_id_seq;
10 CREATE SEQUENCE producto_id_seq
11     START WITH 1 INCREMENT BY 1 MINVALUE 1 NO MAXVALUE NO CYCLE;

```

Listing 2: Secuencias para llaves artificiales

Las tres siguen el mismo patrón. Generan IDs para `DEPENDIENTE`, `CATEGORIA` y `PRODUCTO` respectivamente.

```

1 SELECT sequencename, start_value, increment_by, min_value
2 FROM pg_sequences
3 WHERE schemaname = 'public'
4 ORDER BY sequencename;

```

Listing 3: Verificación de secuencias creadas

Consulta al catálogo interno de PostgreSQL (`pg_sequences`) para confirmar que las cuatro secuencias quedaron correctamente creadas.

Resumen de secuencias

Secuencia	Usada en tabla	Propósito
folio_seq	ORDEN	Número de folio de la orden
dependiente_id_seq	DEPENDIENTE	PK de dependientes
categoria_id_seq	CATEGORIA	PK de categorías
producto_id_seq	PRODUCTO	PK de productos

01_ddl_creacion_tablas.sql — Tablas

Descripción

Crea las 11 tablas del sistema en orden correcto (de independientes a dependientes, respetando las FK). Incluye una sección de limpieza al inicio que elimina las tablas en orden inverso antes de crearlas.

Limpieza previa

```

1 DROP TABLE IF EXISTS DETALLE_ORDEN CASCADE;
2 DROP TABLE IF EXISTS ORDEN CASCADE;
3 DROP TABLE IF EXISTS BEBIDA CASCADE;
4 DROP TABLE IF EXISTS PLATILLO CASCADE;
5 DROP TABLE IF EXISTS PRODUCTO CASCADE;
6 DROP TABLE IF EXISTS CATEGORIA CASCADE;

```

```

7 DROP TABLE IF EXISTS CLIENTE_FACT CASCADE;
8 DROP TABLE IF EXISTS ADMINISTRATIVO CASCADE;
9 DROP TABLE IF EXISTS COCINERO CASCADE;
10 DROP TABLE IF EXISTS MESERO CASCADE;
11 DROP TABLE IF EXISTS DEPENDIENTE CASCADE;
12 DROP TABLE IF EXISTS TELEFONO_EMPLEADO CASCADE;
13 DROP TABLE IF EXISTS EMPLEADO CASCADE;

```

Listing 4: DROP en orden inverso para respetar FKs

El `IF EXISTS` evita error si la tabla no existía. El `CASCADE` elimina también las restricciones que dependan de esa tabla.

Tablas de empleados

```

1 CREATE TABLE EMPLEADO (
2     num_empleado    VARCHAR(10)    NOT NULL,  -- PK alfanumerica (
3     ej. EMP-001)
4     nombre_pila     VARCHAR(40)    NOT NULL,
5     apellido_paterno VARCHAR(40)    NOT NULL,
6     apellido_materno VARCHAR(40)    NOT NULL,
7     rfc              VARCHAR(13)    NOT NULL,  -- unico por empleado
8     fecha_nacimiento DATE           NOT NULL,
9     edad             INTEGER         NOT NULL,
10    sueldo            NUMERIC(10,2)  NOT NULL CHECK (sueldo > 0),
11    foto              BYTEA,          -- opcional, imagen
12                                binaria
13    estado            VARCHAR(40)    NOT NULL,
14    codigo_postal     VARCHAR(5)     NOT NULL,
15    colonia           VARCHAR(100)   NOT NULL,
16    calle             VARCHAR(100)   NOT NULL,
17    numero            VARCHAR(10)    NOT NULL,
18    CONSTRAINT pk_empleado PRIMARY KEY (num_empleado),
19    CONSTRAINT uq_empleado_rfc UNIQUE (rfc)
20 );

```

Listing 5: EMPLEADO — entidad padre

```

1 CREATE TABLE TELEFONO_EMPLEADO (
2     num_empleado VARCHAR(10) NOT NULL,
3     telefono      VARCHAR(15) NOT NULL,
4     CONSTRAINT pk_telefono PRIMARY KEY (num_empleado, telefono),
5     CONSTRAINT fk_telefono_empleado FOREIGN KEY (num_empleado)
6         REFERENCES EMPLEADO (num_empleado)
7         ON DELETE CASCADE ON UPDATE CASCADE
8 );

```

Listing 6: TELEFONO_EMPLEADO — atributo multivaluado

La PK compuesta (`num_empleado`, `telefono`) permite que un empleado tenga varios teléfonos pero no el mismo número dos veces.

```

1 CREATE TABLE DEPENDIENTE (
2     dependiente_id INTEGER NOT NULL DEFAULT nextval('
3         dependiente_id_seq'),
4     num_empleado VARCHAR(10) NOT NULL,
5     nombre VARCHAR(40) NOT NULL,
6     ap_paterno VARCHAR(40) NOT NULL,
7     ap_materno VARCHAR(40) NOT NULL,
8     curp VARCHAR(18) NOT NULL, -- unico por dependiente
9     parentesco VARCHAR(30) NOT NULL,
10    CONSTRAINT pk_dependiente PRIMARY KEY (dependiente_id),
11    CONSTRAINT uq_dependiente_curp UNIQUE (curp),
12    CONSTRAINT fk_dependiente_empleado FOREIGN KEY (num_empleado)
13        REFERENCES EMPLEADO (num_empleado)
14        ON DELETE CASCADE ON UPDATE CASCADE
15 );

```

Listing 7: DEPENDIENTE — familiares del empleado

```

1 CREATE TABLE COCINERO (
2     num_empleado VARCHAR(10) NOT NULL,
3     especialidad VARCHAR(100) NOT NULL,
4     CONSTRAINT pk_cocinero PRIMARY KEY (num_empleado),
5     CONSTRAINT fk_cocinero_empleado FOREIGN KEY (num_empleado)
6         REFERENCES EMPLEADO (num_empleado)
7         ON DELETE CASCADE ON UPDATE CASCADE
8 );
9
10 CREATE TABLE MESERO (
11     num_empleado VARCHAR(10) NOT NULL,
12     horario VARCHAR(50) NOT NULL,
13     CONSTRAINT pk_mesero PRIMARY KEY (num_empleado),
14     CONSTRAINT fk_mesero_empleado FOREIGN KEY (num_empleado)
15         REFERENCES EMPLEADO (num_empleado)
16         ON DELETE CASCADE ON UPDATE CASCADE
17 );
18
19 CREATE TABLE ADMINISTRATIVO (
20     num_empleado VARCHAR(10) NOT NULL,
21     rol VARCHAR(50) NOT NULL,
22     CONSTRAINT pk_administrativo PRIMARY KEY (num_empleado),
23     CONSTRAINT fk_administrativo_empleado FOREIGN KEY (num_empleado)
24         REFERENCES EMPLEADO (num_empleado)
25         ON DELETE CASCADE ON UPDATE CASCADE
26 );

```

Listing 8: Subtipos de EMPLEADO

Cada subtipo solo agrega el atributo diferenciador. La FK con ON DELETE CASCADE garantiza que si se borra un empleado, sus registros de subtipo desaparecen también.

Tablas de productos

```

1 CREATE TABLE CATEGORIA (
2     categoria_id INTEGER NOT NULL DEFAULT nextval('categoria_id_seq
3     '),
4     nombre        VARCHAR(40) NOT NULL,
5     descripcion   TEXT        NOT NULL,
6     CONSTRAINT pk_categoria PRIMARY KEY (categoria_id)
7 );
8
9 CREATE TABLE PRODUCTO (
10     producto_id  INTEGER        NOT NULL DEFAULT nextval('
11     producto_id_seq'),
12     nombre        VARCHAR(40)    NOT NULL,
13     descripcion   TEXT           NOT NULL,
14     precio        NUMERIC(8,2)   NOT NULL CHECK (precio > 0),
15     disponibilidad BOOLEAN       NOT NULL DEFAULT TRUE,
16     receta        TEXT           NOT NULL,
17     categoria_id  INTEGER        NOT NULL,
18     tipo          VARCHAR(10)    NOT NULL CHECK (tipo IN ('platillo
19     ', 'bebida')),
20     CONSTRAINT pk_producto PRIMARY KEY (producto_id),
21     CONSTRAINT fk_producto_categoria FOREIGN KEY (categoria_id)
22         REFERENCES CATEGORIA (categoria_id)
23         ON DELETE RESTRICT ON UPDATE CASCADE
24 );

```

Listing 9: CATEGORIA y PRODUCTO

El campo tipo es un discriminador que complementa a las tablas subtipo PLATILLO y BEBIDA. disponibilidad controla si el producto puede pedirse — los triggers lo validan al insertar en DETALLE_ORDEN.

```

1 CREATE TABLE PLATILLO (
2     producto_id INTEGER NOT NULL,
3     CONSTRAINT pk_platillo PRIMARY KEY (producto_id),
4     CONSTRAINT fk_platillo_producto FOREIGN KEY (producto_id)
5         REFERENCES PRODUCTO (producto_id)
6         ON DELETE CASCADE ON UPDATE CASCADE
7 );
8
9 CREATE TABLE BEBIDA (
10     producto_id INTEGER NOT NULL,
11     CONSTRAINT pk_bebida PRIMARY KEY (producto_id),
12     CONSTRAINT fk_bebida_producto FOREIGN KEY (producto_id)
13         REFERENCES PRODUCTO (producto_id)
14         ON DELETE CASCADE ON UPDATE CASCADE
15 );

```

Listing 10: Subtipos de PRODUCTO

Tablas de órdenes

```

1 CREATE TABLE CLIENTE_FACT (
2   rfc          VARCHAR(13)  NOT NULL,  -- PK: identificador
3   fiscal
4   nombre_cliente VARCHAR(40)  NOT NULL,
5   apellido_paterno VARCHAR(40) NOT NULL,
6   apellido_materno VARCHAR(40) NOT NULL,
7   fecha_nacimiento DATE      NOT NULL,
8   email         VARCHAR(60)  NOT NULL,
9   estado        VARCHAR(40)  NOT NULL,
10  codigo_postal  VARCHAR(5)   NOT NULL,
11  colonia        VARCHAR(40)  NOT NULL,
12  numero         VARCHAR(10)  NOT NULL,
13  calle          VARCHAR(100) NOT NULL,
14  razon_social   VARCHAR(150) NOT NULL,
15  CONSTRAINT pk_cliente_fact PRIMARY KEY (rfc)
);

```

Listing 11: CLIENTE_FACT — solo clientes con factura

```

1 CREATE TABLE ORDEN (
2   folio          VARCHAR(10)  NOT NULL,  -- asignado por trigger
3   fecha          DATE          NOT NULL DEFAULT CURRENT_DATE,
4   hora           TIME          NOT NULL DEFAULT CURRENT_TIME,
5   total          NUMERIC(10,2) NOT NULL DEFAULT 0.00,
6   num_empleado   VARCHAR(10)  NOT NULL,
7   rfc            VARCHAR(13),           -- NULL si no requiere
8   factura
9   CONSTRAINT pk_orden PRIMARY KEY (folio),
10  CONSTRAINT fk_orden_mesero FOREIGN KEY (num_empleado)
11    REFERENCES MESERO (num_empleado)
12    ON DELETE RESTRICT ON UPDATE CASCADE,
13  CONSTRAINT fk_orden_cliente FOREIGN KEY (rfc)
14    REFERENCES CLIENTE_FACT (rfc)
15    ON DELETE SET NULL ON UPDATE CASCADE
);

```

Listing 12: ORDEN — una orden por mesero

El ON DELETE SET NULL en la FK del cliente permite que si se borra un cliente la orden siga existiendo, solo sin asociación de factura.

```

1 CREATE TABLE DETALLE_ORDEN (
2   folio          VARCHAR(10)  NOT NULL,
3   producto_id    INTEGER      NOT NULL,
4   cantidad       INTEGER      NOT NULL CHECK (cantidad > 0),
5   precio_platillo NUMERIC(8,2) NOT NULL CHECK (precio_platillo >
6     0),
7   CONSTRAINT pk_detalle_orden PRIMARY KEY (folio, producto_id)
8   ,
9   CONSTRAINT fk_detalle_orden FOREIGN KEY (folio)
10    REFERENCES ORDEN (folio)

```

```

9      ON DELETE CASCADE ON UPDATE CASCADE,
10     CONSTRAINT fk_detalle_producto FOREIGN KEY (producto_id)
11     REFERENCES PRODUCTO (producto_id)
12     ON DELETE RESTRICT ON UPDATE CASCADE
13 );

```

Listing 13: DETALLE_ORDEN — renglones de la orden

La PK compuesta (folio, producto_id) garantiza que un producto no se repita dentro de la misma orden. El precio_platillo es calculado automáticamente por el trigger — el usuario puede enviar 0.

04_funcion_folio.sql — Función generar_folio

Descripción

Función auxiliar que consume folio_seq y devuelve un folio formateado. Es invocada por el Trigger 1 cada vez que se inserta una orden.

Explicación línea por línea

```

1 CREATE OR REPLACE FUNCTION generar_folio()
2 RETURNS VARCHAR(10) AS $$
3 -- No recibe parametros. Retorna VARCHAR(10),
4 -- suficiente para ORD-001 hasta ORD-9999.
5 DECLARE
6     v_numero INTEGER;      -- guarda el numero de la secuencia
7     v_folio  VARCHAR(10);  -- guarda el folio formateado
8 BEGIN
9     -- Obtiene el siguiente valor de la secuencia.
10    -- Cada llamada incrementa el contador y nunca repite.
11    v_numero := nextval('folio_seq');
12
13    -- Construye el folio en tres pasos:
14    -- 1. v_numero::TEXT -> convierte entero a texto (1 -> '1')
15    -- 2. LPAD(..., 3, '0') -> rellena con ceros ('1' ->
16    --    '001')
17    -- 3. 'ORD-' ||      -> concatena el prefijo ('ORD-001')
18    -- Si el numero supera 3 digitos, LPAD no recorta:
19    -- 1000 -> '1000' -> 'ORD-1000' (7 caracteres, entra en VARCHAR
20    --    (10))
21    v_folio := 'ORD-' || LPAD(v_numero::TEXT, 3, '0');
22
23    RETURN v_folio;
24 END;
25 $$ LANGUAGE plpgsql;

```

Listing 14: Función generar_folio()

Ejemplos de salida

Valor de secuencia	Folio generado
1	ORD-001
50	ORD-050
99	ORD-099
100	ORD-100
1000	ORD-1000

03_triggers.sql — Triggers

Descripción general

Cuatro triggers que automatizan la lógica de negocio sobre ORDEN y DETALLE_ORDEN. En PostgreSQL cada trigger requiere una función separada.

#	Trigger	Tabla	Momento	Acción
1	trg_generar_folio	ORDEN	BEFORE INSERT	Asigna folio automático
2	trg_insertar_detalle	DETALLE_ORDEN	BEFORE INSERT	Valida, calcula subtotal, actualiza total
3	trg_actualizar_detalle	DETALLE_ORDEN	BEFORE UPDATE	Recalcula subtotal y total
4	trg_eliminar_detalle	DETALLE_ORDEN	AFTER DELETE	Descuenta subtotal del total

Trigger 1 — Folio automático

```

1 CREATE OR REPLACE FUNCTION fn_generar_folio_orden()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Asigna el folio generado al registro nuevo antes de
5     -- insertarlo.
6     -- NEW contiene los valores que se van a insertar.
7     NEW.folio := generar_folio();
8     RETURN NEW; -- obligatorio en triggers BEFORE: devuelve el
9     registro modificado
10 END;
11 $$ LANGUAGE plpgsql;
12
13 DROP TRIGGER IF EXISTS trg_generar_folio ON ORDEN;

```

```

13 CREATE TRIGGER trg_generar_folio
14     BEFORE INSERT ON ORDEN -- se ejecuta antes del INSERT
15     FOR EACH ROW           -- una vez por cada fila insertada
16     EXECUTE FUNCTION fn_generar_folio_orden();

```

Listing 15: Función y trigger de folio automático

El usuario no debe proporcionar el folio. Si lo proporciona, el trigger lo sobrescribe.

Trigger 2 — Insertar en DETALLE_ORDEN

```

1 CREATE OR REPLACE FUNCTION fn_insertar_detalle_orden()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     v_disponible BOOLEAN;
5     v_precio     NUMERIC(8,2);
6     v_nombre     VARCHAR(40);
7 BEGIN
8     -- (a) Buscar el producto en catalogo
9     SELECT disponibilidad, precio, nombre
10    INTO v_disponible, v_precio, v_nombre
11   FROM PRODUCTO
12  WHERE producto_id = NEW.producto_id;
13
14     -- Si el SELECT no encontro nada, FOUND = false
15     IF NOT FOUND THEN
16         RAISE EXCEPTION 'El producto con ID % no existe.', NEW.
17             producto_id;
18     END IF;
19
20     -- Validar que este disponible para venta
21     IF v_disponible = FALSE THEN
22         RAISE EXCEPTION 'El producto "%" no esta disponible
23             actualmente.', v_nombre;
24     END IF;
25
26     -- (b) Calcular subtotal: cantidad x precio unitario
27     -- El usuario puede mandar precio_platillo = 0; el trigger lo
28     -- corrige
29     NEW.precio_platillo := NEW.cantidad * v_precio;
30
31     -- (c) Actualizar total de la orden
32     -- COALESCE evita NULL cuando la orden aun no tiene renglones
33     UPDATE ORDEN
34    SET total = (
35        SELECT COALESCE(SUM(precio_platillo), 0) + NEW.
36            precio_platillo
37        FROM DETALLE_ORDEN
38        WHERE folio = NEW.folio
39    )
40   WHERE folio = NEW.folio;

```

```
37
38     RETURN NEW;
39 END;
40 $$ LANGUAGE plpgsql;
```

Listing 16: Función del trigger de inserción

Trigger 3 — Actualizar en DETALLE_ORDEN

```
1 CREATE OR REPLACE FUNCTION fn_actualizar_detalle_orden()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     v_precio NUMERIC(8,2);
5 BEGIN
6     -- Obtener precio actual (puede haber cambiado desde el INSERT)
7     SELECT precio INTO v_precio
8     FROM PRODUCTO
9     WHERE producto_id = NEW.producto_id;
10
11     -- Recalcular subtotal con la nueva cantidad
12     NEW.precio_platillo := NEW.cantidad * v_precio;
13
14     -- Recalcular total de la orden.
15     -- El CASE es necesario: el UPDATE en DETALLE_ORDEN aun no se
16     -- aplico cuando el trigger corre, entonces el renglon
17     -- modificado
18     -- todavia tiene el valor viejo en la tabla. Usamos NEW.
19     -- precio_platillo
20     -- para ese renglon y el valor almacenado para los demas.
21     UPDATE ORDEN
22     SET total = (
23         SELECT COALESCE(SUM(
24             CASE
25                 WHEN folio = NEW.folio AND producto_id = NEW.
26                     producto_id
27                 THEN NEW.precio_platillo -- valor nuevo calculado
28                 ELSE precio_platillo -- valor existente en
29                     tabla
30             END
31         ), 0)
32     FROM DETALLE_ORDEN
33     WHERE folio = NEW.folio
34 )
35 WHERE folio = NEW.folio;
36
37     RETURN NEW;
38 END;
39 $$ LANGUAGE plpgsql;
```

Listing 17: Función del trigger de actualización

Trigger 4 — Eliminar de DETALLE_ORDEN

```

1 CREATE OR REPLACE FUNCTION fn_eliminar_detalle_orden()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     -- Recalcular total excluyendo el renglon eliminado.
5     -- Este trigger es AFTER DELETE, entonces el renglon ya no
6     -- existe
7     -- en la tabla cuando se ejecuta esta funcion.
8     -- OLD contiene los valores del renglon que se elimino.
9     UPDATE ORDEN
10    SET total = (
11        SELECT COALESCE(SUM(precio_platillo), 0)
12        FROM DETALLE_ORDEN
13        WHERE folio = OLD.folio
14              AND producto_id != OLD.producto_id -- excluir el
15              eliminado
16    )
17    WHERE folio = OLD.folio;
18
19    RETURN OLD; -- en triggers AFTER DELETE se retorna OLD
20 END;
21 $$ LANGUAGE plpgsql;
22
23 CREATE TRIGGER trg_eliminar_detalle
24 AFTER DELETE ON DETALLE_ORDEN -- AFTER porque necesitamos que
    ya este borrado
    FOR EACH ROW
    EXECUTE FUNCTION fn_eliminar_detalle_orden();

```

Listing 18: Función del trigger de eliminación

Nota importante: Este trigger usa `AFTER DELETE` a diferencia de los anteriores que usan `BEFORE`. Esto es necesario porque necesita que el registro ya esté eliminado de la tabla para que el `SUM` lo excluya de forma natural.

test_triggers.sql — Casos de Prueba**Descripción**

Siete casos de prueba que verifican el correcto funcionamiento de los cuatro triggers. Incluye datos de prueba, operaciones y resultados esperados.

Datos base

```

1 -- Categoria
2 INSERT INTO CATEGORIA (categoria_id, nombre, descripcion)
3 VALUES (nextval('categoria_id_seq'), 'Comida mexicana', 'Platillos
    tipicos mexicanos');

```

```

4
5  -- Empleado
6  INSERT INTO EMPLEADO (num_empleado, nombre_pila, apellido_paterno,
7      apellido_materno,
8      rfc, fecha_nacimiento, edad, sueldo, estado, codigo_postal,
9      colonia, calle, numero)
10 VALUES ('EMP-001', 'Juan', 'Garcia', 'Lopez',
11     'GALJ900101ABC', '1990-01-01', 34, 8000.00,
12     'CDMX', '06600', 'Juarez', 'Reforma', '100');
13
14 -- Mesero (subtipo de empleado)
15 INSERT INTO MESERO (num_empleado, horario)
16 VALUES ('EMP-001', 'Lunes a Viernes 08:00-16:00');
17
18 -- Productos: 2 disponibles, 1 no disponible
19 INSERT INTO PRODUCTO (producto_id, nombre, descripcion, precio,
20     disponibilidad, receta, categoria_id, tipo)
21 VALUES
22     (nextval('producto_id_seq'), 'Tacos de pastor',
23     'Tacos con carne al pastor', 85.00, TRUE,
24     'Carne de cerdo marinada...', 1, 'platillo'),
25     (nextval('producto_id_seq'), 'Agua de jamaica',
26     'Bebida de flor de jamaica', 25.00, TRUE,
27     'Flor de jamaica, agua, azucar...', 1, 'bebida'),
28     (nextval('producto_id_seq'), 'Enchiladas verdes',
29     'Enchiladas con salsa verde', 95.00, FALSE, -- no disponible
30     'Tortillas, salsa verde...', 1, 'platillo');
31
32 INSERT INTO PLATILLO (producto_id) VALUES (1);
33 INSERT INTO BEBIDA (producto_id) VALUES (2);
34 INSERT INTO PLATILLO (producto_id) VALUES (3);

```

Listing 19: Inserción de datos de prueba

Pruebas

```

1  INSERT INTO ORDEN (fecha, hora, num_empleado)
2  VALUES (CURRENT_DATE, CURRENT_TIME, 'EMP-001');
3  -- El usuario NO proporciona folio; el trigger lo asigna.
4
5  SELECT folio, fecha, hora, total FROM ORDEN;
6  -- Resultado esperado: folio = 'ORD-001', total = 0.00

```

Listing 20: Prueba 1: Folio automático

```

1  -- 2 tacos de pastor a $85 c/u = $170
2  -- Se envia precio_platillo = 0; el trigger lo corrige
3  INSERT INTO DETALLE_ORDEN (folio, producto_id, cantidad,
4      precio_platillo)
5  VALUES ('ORD-001', 1, 2, 0);

```

```

6 SELECT * FROM DETALLE_ORDEN WHERE folio = 'ORD-001';
7 -- Resultado esperado: precio_platillo = 170.00
8
9 SELECT folio, total FROM ORDEN WHERE folio = 'ORD-001';
10 -- Resultado esperado: total = 170.00

```

Listing 21: Prueba 2: INSERT con producto disponible

```

1 -- 1 agua de jamaica a $25
2 INSERT INTO DETALLE_ORDEN (folio, producto_id, cantidad,
3   precio_platillo)
4 VALUES ('ORD-001', 2, 1, 0);
5
6 SELECT folio, total FROM ORDEN WHERE folio = 'ORD-001';
7 -- Resultado esperado: total = 195.00 ($170 + $25)

```

Listing 22: Prueba 3: Acumulación de total

```

1 -- Cambiar tacos de 2 a 3 unidades: 3 x $85 = $255
2 UPDATE DETALLE_ORDEN SET cantidad = 3
3 WHERE folio = 'ORD-001' AND producto_id = 1;
4
5 SELECT * FROM DETALLE_ORDEN WHERE folio = 'ORD-001';
6 -- Resultado esperado: precio_platillo de tacos = 255.00
7
8 SELECT folio, total FROM ORDEN WHERE folio = 'ORD-001';
9 -- Resultado esperado: total = 280.00 ($255 + $25)

```

Listing 23: Prueba 4: UPDATE de cantidad

```

1 -- Eliminar agua de jamaica
2 DELETE FROM DETALLE_ORDEN
3 WHERE folio = 'ORD-001' AND producto_id = 2;
4
5 SELECT folio, total FROM ORDEN WHERE folio = 'ORD-001';
6 -- Resultado esperado: total = 255.00 (se restan los $25)

```

Listing 24: Prueba 5: DELETE de un renglón

```

1 -- Enchiladas verdes tiene disponibilidad = FALSE
2 -- El bloque DO captura la excepcion sin abortar el script
3 DO $$
4 BEGIN
5     INSERT INTO DETALLE_ORDEN (folio, producto_id, cantidad,
6       precio_platillo)
7     VALUES ('ORD-001', 3, 1, 0);
8
9     -- Esta linea solo se ejecuta si el trigger falla en lanzar el
10    error
11    RAISE NOTICE 'ERROR: El trigger no lanzo la excepcion esperada'
12    ;
13 EXCEPTION
14 WHEN OTHERS THEN

```



```

12      -- SQLERRM contiene el mensaje de error del trigger
13      RAISE NOTICE 'OK: Excepcion capturada correctamente -> %',
        SQLERRM;
14  END;
15  $$;
16  -- Resultado esperado: NOTICE con el mensaje del trigger

```

Listing 25: Prueba 6: Producto no disponible

```

1  INSERT INTO ORDEN (fecha, hora, num_empleado)
2  VALUES (CURRENT_DATE, CURRENT_TIME, 'EMP-001');
3
4  SELECT folio FROM ORDEN ORDER BY folio;
5  -- Resultado esperado: ORD-001 y ORD-002

```

Listing 26: Prueba 7: Segunda orden con folio incremental

Resumen de pruebas

Prueba	Operación	Resultado esperado
1	INSERT en ORDEN	folio = ORD-001, total = \$0.00
2	INSERT 2 tacos (\$85)	subtotal = \$170, total = \$170
3	INSERT 1 jamaica (\$25)	total = \$195
4	UPDATE tacos a 3 uds.	subtotal = \$255, total = \$280
5	DELETE jamaica	total = \$255
6	INSERT no disponible	excepción controlada
7	Segunda orden	folios ORD-001 y ORD-002

Diferencias entre la versión Oracle y PostgreSQL

Aspecto	Oracle 23ai (00_ejecutar_todo)	PostgreSQL (01-04)
Tipo binario	BLOB	BYTEA
Variables en trigger	:NEW.campo, :OLD.campo	NEW.campo, OLD.campo
ON UPDATE CASCADE	No soportado (omitido)	Sí soportado
Estructura trigger	Código directo en el trigger	Requiere función separada (fn...)
Problema tabla mutante	Solución incremental (total = total + subtotal)	No aplica; se usa SUM() completo
Tipo hora	DATE (incluye hora) con DEFAULT SYSDATE	TIME separado con DEFAULT CURRENT_TIME
Manejo de errores	RAISE_APPLICATION_ERROR(-20001, 'msg')	RAISE, EXCEPTION 'msg'
Verificar errores	SHOW ERRORS	NOT FOUND (variable implícita)